- Edy Lane Marinez Rojas - 202510939-.
- Isabella Casallas Abril - 202510253

Proyecto 3: ejecución de programas Objectilang

1. Resumen

En el proyecto se hizo lo siguiente:

1. Se corrigió el resaltado de sintaxis que se señaló en la revisión del Proyecto 2.
2. Se agregaron pruebas en archivos .ol y un módulo que las corre todas.
3. Se creó un generador que traduce cada programa .ol a Java, lo ejecuta y muestra su salida en la consola.
4. Se agregó al plugin de VS Code un botón para generar y ejecutar el archivo abierto.
5. Se instaló TypePal y se construyó un chequeador de tipos (módulo Check.rsc) que verifica que los elementos usados en el programa existan (objetos, variables, herencia) y que los tipos declarados coincidan con los valores.

La gramática acepta los mismos programas que en el Proyecto 2, y el AST no cambió.

2. Corrección del resaltado de sintaxis

Problema

Solo object, extends, method, val y var tenían categoría de resaltado. Esas categorías estaban en la declaración keyword Reservada, que solo impide usar una palabra reservada como identificador y nunca aparece en el árbol de un programa, así que no coloreaba nada. Además, las producciones de métodos y objetos tenían @category="method" y @category="class" sobre toda la regla, y el editor pintaba todo su contenido de ese color.

Solución

La categoría se pone en la producción donde aparece cada símbolo. El editor le da a cada literal la categoría de la producción más cercana que tenga una; los identificadores y los números conservan la suya.

Elementos	**Categoría**
object, extends, method, main, val, var, new, override, if, then, else, for, in, and, or, neg	keyword

Elementos	**Categoría**
true, false	keyword
String, Integer, Boolean, Char	type
** * / % + - < > <= >= <> = -> => y la varianza + / -	operator
Literales Char ('a') y String	string
Comentarios //	comment
Puntuación ( ) { } [ ] , : .	sin categoría

Ejemplo, en Syntax.rsc:

```
syntax Expresion
  = @category="keyword" ifConElse: "if" !>> [a-zA-Z0-9] Expresion "then" !>> [a-zA-Z0-9] Expresion "else" !>> [a-zA-Z0-9] Expresion
  ...
syntax Tipo
  = @category="type" tipoString: "String" !>> [a-zA-Z0-9]
  ...
  > left ( @category="operator" suma: Operacion "+" Operacion | ... )
```

También se agregó la restricción !>> [a-zA-Z0-9] a then, else, in, and y or. Sin ella, if a thenb else c se aceptaba como if a then b else c.

Limitación

Unos pocos símbolos toman la categoría de su producción y salen como keyword: <- en el for, el = de Objeto.metodo = method(...) y del override, y el : de los parámetros del constructor. Para separarlos habría que crear un no terminal por símbolo, y eso obligaría a cambiar el AST.

3. Pruebas

Se agregaron programas .ol en la carpeta instance y el módulo Pruebas.rsc, que los parsea e implota todos (import Pruebas; main();). Los archivos que empiezan por falla_ deben dar error de sintaxis; los demás deben parsear e implotar sin errores ni ambigüedades. Resultado: 26 de 26.

Además, el chequeador de tipos tiene su propio juego de pruebas (tipos_*.ol), que corre main() de Check.rsc (ver sección 5).

Archivos	Qué prueban
prueba1 a prueba6	Las pruebas de sintaxis del Proyecto 2
enunciado1 a enunciado4	Los cuatro ejemplos del enunciado, completados con un main cuando hacía falta
ejec01 a ejec07	Programas con salida conocida: aritmética, if, métodos, herencia, campos, override, tuplas y for
cadenas	Interpolación con $nombre y ${expresión}, escapes y string vacío
resaltado	Un elemento de cada categoría de resaltado
errorEjec_operador, errorEjec_sinCuerpo, errorEjec_variable	Errores de ejecución: operador sin semántica, método sin cuerpo y variable no definida
falla_comaFinal, falla_reservada, falla_sinMain, falla_ifSinThen	Programas que el parser debe rechazar

Cada programa ejecutable trae en un comentario la salida esperada.

4. Generador de Java

El módulo Generador.rsc traduce el AST de un programa a una clase Java, la escribe en instance/output/OL_<archivo>.java y la ejecuta. La salida del programa se muestra en la consola y se guarda en instance/output/OL_<archivo>.salida.txt.

Cómo funciona

1. generator1(archivo) parsea el .ol, lo implota al AST con implodePrograma y llama a generar.
2. generar está definida una vez por cada constructor del AST. Rascal escoge la versión que corresponde según el patrón del parámetro. Por ejemplo:

```
str generar(suma(Operacion a, Operacion b)) = binaria("suma", a, b);
str generar(litEntero(int n)) = "<n>";
```

3. El código Java se arma con plantillas de strings de Rascal y se escribe con writeFile.
4. ejecutar(archivo) corre el .java con java Archivo.java, que compila y ejecuta en un solo paso, e imprime lo que el programa muestra.

Traducción

Objectilang	**Java**
object Dog(val n: T) extends Animal	static class Dog extends Animal, con un campo final por cada parámetro del constructor
Dog.talk = method(): R { ... }	Método talk de la clase Dog. Devuelve el valor de la última sentencia del bloque
Método sin cuerpo	Método que lanza el error "el metodo ... no tiene cuerpo"
new Animal { override hablar(): R = { ... } }	Clase anónima que sobrescribe el método
val x = e y var x = e	final var x = e;
if c then a else b	Operador ? : de Java, o un if cuando se usa como sentencia
for ((k, v) <- t) e	for de Java sobre los elementos de la tupla
Tupla (a, b)	Object[]
"Hola $n y ${e}"	Concatenación de las partes del string
print, println y los operadores	Funciones de una clase OL que va al final de cada archivo generado

Los valores se manejan como Object; solo los objetos definidos en el programa conservan su tipo en Java, para poder llamar sus métodos. La clase OL revisa en ejecución que cada operador reciba valores válidos y, si no, detiene el programa con Error de ejecucion: ....

Caso	**Comportamiento**
+	Suma enteros; si un lado es String o Char, concatena
/ y %	División entera. Entre cero dan error
**	Asocia a la derecha. Con exponente negativo da error
in	Busca un valor en una tupla o un texto en un String

Caso	Comportamiento
-> y =>	Error "operador sin semantica definida"
Un objeto dentro de un texto	Se muestra con el nombre de su objeto (Dog)

Ejemplo

Programa ejec05_campos.ol:

```
object Animal
object Dog extends Animal
object Caja[+A](val item: A)
Caja.sacar = method(): A { item }

method main {
  val k: Caja[Dog] = new Caja(new Dog)
  println(k.item)
  println(k.sacar())
}
```

Java generado (sin la clase OL):

```
public class OL_ejec05_campos {
  static class Animal { Animal() { } }
  static class Dog extends Animal { Dog() { } }
  static class Caja {
    final Object item;
    Caja() { this.item = null; }
    Caja(Object item$) { this.item = (Object) item$; }
    public Object sacar() { return item; }
  }

  public static void main(String[] args$) {
    try {
      final var k = new Caja(new Dog());
      OL.println(new Object[] {k.item});
      OL.println(new Object[] {k.sacar()});
    } catch (RuntimeException e$) {
      OL.reportar(e$);
    }
  }
}
```

Salida:

```
Dog
Dog
```

Resultados

Programa	**Salida**
ejec01_aritmetica	7 y division 3 resto 1 pot 512
ejec02_condicional	grande
ejec03_metodos	Cats meows dos veces
ejec04_herencia	Sonido de animal
ejec05_campos	Dog dos veces
ejec06_override	sonido sobrescrito
ejec07_tuplas_for	Movie: ana, Rating: 5 y Movie: bob, Rating: 3
enunciado1	Print the line of animal dogs: Dog
enunciado2	Cats meows
enunciado3	Preparing specialized food for an animal... y Resulting food type: DogFood
enunciado4	7 y las dos líneas de películas
cadenas	Hola Ana, tienes 21 años, Hola Ana, Anidado: edad 20, Escapes: $5, " y \
errorEjec_operador	Error de ejecucion: operador sin semantica definida: ->
errorEjec_sinCuerpo	Error de ejecucion: el metodo transform no tiene cuerpo
errorEjec_variable	El Java no compila: cannot find symbol ... variable y. La verificación de tipos lo detecta antes de generar

5. Generador de Tipos (TypePal):

Se construyó un chequeador de tipos en el módulo nuevo Check.rsc, usando TypePal. Revisa dos cosas sobre cada programa, las cuales son la regla de existencia (que todo lo que se usa esté definido) y la correspondencia de tipos (que los tipos encajen). Cubre tanto los tipos primitivos como los objetos, la herencia y los genéricos.

Instalacion de TypePal:
Se agregó la dependencia de TypePal al pom.xml. La versión 0.14.2 resultó incompatible con Rascal 0.43.0 (al construir el colector lanzaba el error "Cannot find loc on tree"), así que se usó **TypePal 0.17.0**, que sí es compatible. La versión compatible se tomó del pom.xml de otro grupo, como permite el enunciado; es el único elemento que se reutilizó.

Correspondencia de tipos:
Literales, operadores (cada uno con su regla), las ramas del if, el for sobre tuplas, las tuplas, las declaraciones con tipo (val x: T = e), los argumentos de new y de las llamadas a métodos, el valor de retorno de los métodos y de los override, y los subtipos con herencia y varianza de los genéricos (+A covariante, -A contravariante).

Regla de existencia:
El padre de extends, los tipos de los campos y de los métodos, los parámetros genéricos, el objeto de Objeto.metodo, el método que se sobrescribe con override, y los campos y métodos usados con x.campo y x.metodo(...). Además detecta objetos, campos y métodos duplicados y herencia cíclica.

Funcionamiento de Check.rsc:
TypePal recorre el árbol con funciones collect y usa cinco operaciones:

- **define** registra un nombre (objeto, variable o parámetro).
- **use** comprueba que un nombre exista; si no tiene define, reporta Undefined object/variable.
- **fact** fija un tipo ya conocido (un literal, un campo visible del objeto).
- **calculate** calcula el tipo de una expresión a partir de sus partes (operadores, if, tuplas, acceso .campo/.metodo, y new —incluso infiriendo los argumentos de tipo, como en new Caja(new Dog)).
- **require** exige una condición de tipo (que la condición del if sea Boolean, que el valor de una declaración sea subtipo del tipo declarado, que el cuerpo de un método sea subtipo de su tipo de retorno, que los argumentos encajen).

Antes de recolectar, construirTabla arma un mapa de los objetos y métodos del programa, para poder revisar usos que aparecen **antes** de su definición. La relación de subtipos (esSubtipo) sube por la cadena de herencia y respeta la varianza de los genéricos. Una configuración de TypePal (isAcceptableSimple) hace que una variable solo se pueda usar después de declararse, mientras que los objetos y parámetros se ven en cualquier parte.

Genéricos y campos:
Los nombres de los parámetros genéricos (la A de Caja[+A]) y de los campos y parámetros de métodos se recolectan antes de chequear y se excluyen de la verificación de existencia, para no marcarlos como inexistentes.

Dentro del cuerpo de un método, sus campos y genéricos quedan visibles.

Entrada y pruebas:
revisarArchivo(archivo) parsea el .ol, arma la tabla y llama a collectAndSolve. errores(archivo) imprime solo los errores. Cada archivo de prueba tipos_*.ol trae una línea // errores: N con el número de errores esperado; main() los corre todos y reporta el resultado (29 de 29). Los programas válidos salen sin errores; los inválidos reportan el error esperado (p. ej. una herencia de un objeto inexistente da Undefined object, y val x: Integer = true da “El valor es Boolean pero x se declaró como Integer”).

Hallazgo enunciado 3:
El chequeador marca errores en enunciado3. El programa crea la instancia con new Transformer[...] pero declara la variable con el tipo Transform (sin la “r” final): son dos nombres distintos, así que Transform no existe como objeto y los usos que dependen de él fallan. Es un error real del ejemplo, no del chequeador. La versión con el nombre bien escrito está en tipos_bien_objetos.ol y sale sin errores.

6. Plugin

Plugin.rsc agrega un botón **Generar Java y ejecutar** encima de cada archivo .ol. Al hacer clic genera el Java, lo ejecuta y abre dos archivos: el .java generado y el .salida.txt con lo que imprimió el programa. La salida también aparece en el panel Output del servidor de Rascal.

7. Cómo ejecutar

Desde una terminal de Rascal en la carpeta objectilang:

Acción	**Comando**
Correr las pruebas de sintaxis	import Pruebas; y main();
Generar y ejecutar un programa	import Generador; y `ejecutar(
Generar y ejecutar todos los ejemplos	import Generador; y main();
Ejecutar a mano un Java generado	java instance/output/OL_enunciado1.java, en una terminal del sistema
Correr todas las pruebas de tipos	import Check; y main();

En VS Code se puede correr de la siguiente forma: correr main de Plugin.rsc con Run in new Rascal terminal, abrir un .ol y pulsar Generar Java y ejecutar.

Para el chequeo de tipos: en una terminal de Rascal, import Check; y luego errores(|project://objectilang/instance/<archivo>.ol|); para ver los errores de tipo y de nombres de un programa.

8. Archivos modificados y nuevos

Archivo	**Cambio**
Syntax.rsc	Categorías de resaltado y restricciones !>> en then, else, in, and y or
Generador.rsc	Nuevo: generador de Java y ejecución
Plugin.rsc	Botón para generar y ejecutar
Pruebas.rsc	Nuevo: corre las pruebas de la carpeta instance
Implode.rsc	Se quitaron sus funciones de prueba, que ahora están en Pruebas.rsc
instance/*.ol	Programas de prueba nuevos (sección 3), incluye los archivos de prueba del chequeador (tipos_*.ol).
.gitignore	Ignora la carpeta generada instance/output
Check.rsc	Chequeador de tipos completo, es decir, existencia, correspondencia de tipos, subtipos con herencia y varianza, duplicados y herencia cíclica.
pom.xml	Se agregó la dependencia de TypePal (versión 0.17.0).

AST.rsc, Parser.rsc y Cadenas.rsc no cambiaron su comportamiento.

Graciasss :D